

Relatório do Projeto

Desafio Técnico — Desenvolvedor Fullstack Júnior — Watch Brasil

Descrição do Projeto

Studio Flow é um sistema de gerenciamento de tarefas colaborativo, desenvolvido como resposta ao desafio técnico de Desenvolvedor Fullstack Júnior da Watch Brasil. A aplicação permite que usuários criem contas, organizem tarefas em categorias, colaborem entre si e acompanhem o progresso do trabalho por meio de relatórios agregados.

O projeto foi construído em Node.js, seguindo uma arquitetura em camadas, e implantado em ambiente serverless real na AWS Lambda — atendendo integralmente ao objetivo proposto pelo desafio. Além dos requisitos obrigatórios (CRUD completo, frontend em Vue.js, testes automatizados, banco relacional, documentação OpenAPI e deploy serverless), todos os itens desejáveis também foram implementados: autenticação com JWT, uso de contêiner (Docker) e instrumentação de rastreamento com OpenTelemetry e Jaeger.

Propósito da aplicação

O Studio Flow é um sistema de gerenciamento de tarefas colaborativo. O propósito é permitir que uma equipe: Crie contas e faça login de forma segura.

Organize tarefas em categorias (com prioridade, status e prazo).

Colabore: o dono de uma tarefa pode adicionar colegas por e-mail pra trabalharem juntos nela.

Acompanhe o progresso através de relatórios (quantas tarefas estão pendentes, em andamento, concluídas, atrasadas, etc.).

Arquitetura do Projeto

O backend segue uma arquitetura em camadas, organizada por módulo de domínio (autenticação, usuários, categorias, tarefas, relatórios). Cada módulo é dividido em quatro camadas: rota (recebe a requisição e valida os dados de entrada), controller (traduz a requisição HTTP), service (aplica a regra de negócio) e repository (executa as consultas no banco de dados).

Um único arquivo central de aplicação (app.js) concentra toda a configuração do Express — rotas, middlewares, CORS e documentação — e é reaproveitado por dois pontos de entrada diferentes: um servidor HTTP tradicional, usado no desenvolvimento local, e um adaptador para AWS Lambda, usado em produção. Isso elimina qualquer duplicação de lógica entre os dois ambientes de execução.

Arquitetura de implantação

A infraestrutura de produção é inteiramente gerenciada, sem servidores dedicados:

- **AWS Lambda + API Gateway** — executam o backend, empacotado como um pacote único e otimizado.
- **AWS S3 + CloudFront** — hospedam o frontend como site estático, com HTTPS.
- **Banco de dados PostgreSQL na nuvem** — acessado por uma conexão compatível com ambientes serverless.
- **Jaeger** — recebe os dados de rastreamento das requisições, quando habilitado.

Segurança

- Senhas protegidas por hash (bcryptjs), nunca armazenadas em texto simples.
- Autenticação sem estado, usando token assinado (JWT) — não depende de sessão guardada no servidor.
- Cada consulta de tarefa confere se o usuário é o dono ou um colaborador antes de liberar o dado.
- Cabeçalhos de segurança HTTP aplicados a todas as respostas (Helmet).
- Validação estrita de todos os dados recebidos pela API.

Configuração (variáveis de ambiente)

Variável	Finalidade
DATABASE_URL	Connection string do PostgreSQL
JWT_SECRET / JWT_EXPIRES_IN	Chave de assinatura e validade do token de autenticação
PORT	Porta do servidor local (padrão 3000)
FRONTEND_URL	Origem permitida para CORS
OTEL_ENABLED / OTEL_EXPORTER_OTLP_ENDPOINT	Ativação e destino do rastreamento (OpenTelemetry/Jaeger)
AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY	Credenciais usadas somente para o deploy (não lidas pela aplicação em si)

O que foi Desenvolvido

- **Contas de usuário** — cadastro e login por e-mail e senha, com senha protegida por hash.
- **Categorias** — criação, edição, listagem e exclusão, com cor personalizável. Toda conta nova recebe automaticamente cinco categorias de exemplo, seguindo um fluxo de produção de conteúdo.
- **Tarefas** — criação, edição, listagem e exclusão, com título, descrição, status, prioridade, prazo e categoria.
- **Colaboração entre usuários** — o dono de uma tarefa pode adicionar ou remover colaboradores por e-mail. Colaboradores podem visualizar e editar a tarefa, mas não podem excluí-la nem gerenciar outros colaboradores.
- **Relatórios** — resumo com total de tarefas, distribuição por status e prioridade, tarefas atrasadas, taxa de conclusão, e distribuição por categoria — apresentados com gráficos.
- **Busca e filtros** — filtragem de tarefas por status, categoria e busca por título.
- **Identidade visual própria** — paleta de cores e nome de marca ("Studio Flow") desenvolvidos especificamente para o projeto, com interface responsiva para celular.

Como o Projeto foi Construído

- **Etapas 1 — Planejamento** — definição da stack tecnológica e da estrutura de pastas antes de iniciar a implementação, com alinhamento explícito sobre quais funcionalidades extras seriam incluídas.
- **Etapas 2 — Construção do backend** — implementado módulo por módulo (autenticação, categorias, tarefas, relatórios), sempre seguindo a mesma estrutura em camadas, com testes automatizados escritos junto de cada módulo.
- **Etapas 3 — Construção do frontend** — iniciada somente após o backend estar funcional e testado, com as telas de login, cadastro, painel de tarefas, categorias e relatórios.
- **Etapas 4 — Verificação com infraestrutura real** — além dos testes automatizados, foi realizada uma verificação manual completa com um banco de dados PostgreSQL real, o que revelou um erro que os testes automatizados não conseguiriam detectar (descrito na seção de Desafios).
- **Etapas 5 — Uso de contêiner (Docker)** — criação do ambiente Docker para a API, o que revelou um problema de conectividade de rede com o banco de dados.
- **Etapas 6 — Deploy real na AWS Lambda** — configuração de credenciais, primeira tentativa de deploy (que revelou um problema de empacotamento), correção, e novo deploy validado com testes reais contra a API pública.

- **Etapa 7 — Publicação do frontend** — build do frontend apontando para a API real, publicação em um bucket S3, e configuração de permissões entre domínios (CORS).
- **Etapa 8 — Ajuste de segurança de acesso (HTTPS)** — identificado que o endereço padrão do S3 não oferece conexão segura (HTTPS), o que impedia o acesso pelo link em determinados aplicativos e navegadores. Corrigido com a adição de uma camada de distribuição de conteúdo (CloudFront) na frente do site.
- **Etapa 9 — Refinamento a partir de feedback** — ajustes de identidade visual, paleta de cores e adaptação para celular, cada mudança validada visualmente antes de ser publicada novamente.
- **Etapa 10 — Controle de versão** — histórico de mudanças organizado por assunto, com verificação constante para nenhuma senha ou credencial ser enviada ao repositório por engano.

Desafios Encontrados e Soluções

Falha de conexão dentro do Docker

O endereço de conexão "direta" do banco de dados só funcionava por um tipo de rede (IPv6) que o Docker não conseguia usar.

Como foi resolvido: Substituição pela conexão em modo "pooler" do mesmo banco, compatível com IPv4.

Deploy na nuvem falhava com erro de excesso de arquivos abertos

O processo padrão de empacotamento tentava incluir todos os arquivos do projeto no pacote de deploy, incluindo ferramentas usadas apenas localmente — mais de 40 mil arquivos ao todo.

Como foi resolvido: Troca da forma de empacotamento para uma que inclui somente o código realmente necessário em produção, reduzindo o pacote final para cerca de 45 MB.

Link do site não abria em alguns aplicativos (WhatsApp, navegadores de celular)

O endereço padrão de hospedagem de site do S3 não oferece conexão segura (HTTPS), e diversos aplicativos modernos recusam ou alertam sobre conexões sem essa proteção.

Como foi resolvido: Adição de uma camada de distribuição de conteúdo (CloudFront) na frente do site, que fornece um endereço com conexão segura.

Uso de Inteligência Artificial

Contei com o uso do Claude Code, IA, assistente de inteligência artificial da Anthropic voltado para engenharia de software, utilizado como ferramenta de apoio à programação. É importante documentar esse uso com transparência, já que representa uma habilidade cada vez mais relevante no mercado de tecnologia: a capacidade de direcionar, validar e depurar soluções construídas com apoio de inteligência artificial, em vez de aceitar resultados sem revisão crítica.

- **Direcionamento claro e decisões rápidas** — cada etapa do projeto foi conduzida com prioridades bem definidas — por exemplo, quais tecnologias usar, ou quais funcionalidades extras do desafio implementar — evitando indecisão e retrabalho.
- **Validação real, não aceitação automática** — nenhuma etapa foi considerada pronta apenas por o código ter sido escrito. Cada entrega foi conferida de verdade: testes automatizados executados, aplicação rodando de fato, banco de dados real, e a API funcionando na nuvem — processo que revelou e permitiu corrigir erros genuínos antes da entrega.
- **Decisões técnicas conscientes** — ao longo do projeto, diversas escolhas exigiram comparar alternativas — como usar SQL direto ou um ORM, ou qual biblioteca de autenticação usar — e cada uma foi resolvida com critérios claros (segurança, compatibilidade com o ambiente de nuvem, simplicidade), documentados neste relatório.
- **Ajustes a partir de feedback real** — mudanças de identidade visual e de usabilidade partiram de observações diretas sobre a experiência de uso, traduzidas em alterações específicas.

- **Depuração de problemas reais** — os problemas descritos na seção de Desafios foram identificados a partir de evidências concretas (mensagens de erro, comportamento da aplicação), sendo o papel principal interpretar essas evidências corretamente e decidir a correção adequada.

Em resumo, a inteligência artificial foi utilizada como uma ferramenta que acelera a execução, mas as decisões de arquitetura, os critérios de aceitação de cada etapa e a interpretação dos problemas encontrados foram conduzidos ativamente ao longo de todo o processo.

Testes e Qualidade

Foram escritos testes automatizados com Jest (testes unitários das regras de negócio) e Supertest (testes de integração das rotas HTTP), cobrindo utilitários de segurança, as regras de autorização (como a que garante que só o dono de uma tarefa pode gerenciar colaboradores), e os fluxos principais de autenticação e de tarefas. Nesses testes, o acesso ao banco de dados é simulado, o que garante execução rápida e resultados sempre consistentes.

Essa abordagem tem um limite conhecido: como o banco é simulado, os testes não executam SQL de verdade, e por isso não detectaram o erro de conversão de tipo descrito na seção de Desafios. Para compensar esse limite, foram feitas verificações manuais completas contra um banco de dados PostgreSQL real, tanto localmente quanto após cada publicação em nuvem — o que efetivamente encontrou e permitiu corrigir esse problema antes da entrega final.

Como Executar o Projeto

Pré-requisitos

- Node.js 20 ou superior.
- Uma instância PostgreSQL acessível (recomendado: Neon ou Supabase, camada gratuita, prontos para uso serverless).
- Docker (opcional, para rodar a API em contêiner junto com o Jaeger).
- Conta AWS com credenciais configuradas (apenas se for reproduzir o deploy real na Lambda).

Configuração do banco de dados

- Criar um projeto gratuito no Neon ou no Supabase.
- Copiar a connection string do banco (no Neon, preferir a variante "pooled"; no Supabase, usar a connection string do "Connection Pooler" — necessária para compatibilidade com AWS Lambda, que só tem saída IPv4).
- Colar essa string na variável DATABASE_URL do arquivo .env do backend.

Backend (API)

```
cd backend
cp .env.example .env
# edite o .env: preencha DATABASE_URL e gere um JWT_SECRET forte
# ex.: openssl rand -hex 32
npm install
npm run migrate:up      # cria as tabelas: users, categories, tasks, task_collaborators
npm run dev             # inicia em http://localhost:3000
```

Após subir, é possível verificar em: GET <http://localhost:3000/api/health> (healthcheck) e GET <http://localhost:3000/docs> (Swagger UI interativo).

Frontend (interface)

```
cd frontend
cp .env.example .env
# ajuste VITE_API_URL se a API não estiver em localhost:3000/api
npm install
```

```
npm run dev # inicia em http://localhost:5173
```

Fluxo de uso: criar uma conta em Cadastro (já vem com 5 categorias padrão de um fluxo de produção de conteúdo), fazer login, criar tarefas (associando categoria, prioridade e prazo), adicionar colaboradores por e-mail numa tarefa, e acompanhar o progresso em Relatórios.

Rodando com Docker

Sobe a API e o Jaeger (rastreamento) juntos, a partir da raiz do repositório:

```
cd backend && npm run migrate:up # rodar as migrations uma vez, fora do container
cd ..
docker compose up --build
```

API em <http://localhost:3000>, painel do Jaeger em <http://localhost:16686>.

Testes automatizados

```
cd backend
npm test # testes unitários e de integração (Jest)
npm run test:coverage # com relatório de cobertura
```

Deploy real (AWS Lambda + S3/CloudFront)

Pré-requisito: um usuário/role IAM com permissão para Lambda, API Gateway, IAM, CloudFormation e CloudWatch Logs (não precisa ter o AWS CLI instalado).

```
cd backend
export DATABASE_URL="postgresql://..."
export JWT_SECRET="..."
export AWS_ACCESS_KEY_ID="..."
export AWS_SECRET_ACCESS_KEY="..."
npm run deploy # cria os recursos na conta AWS e imprime a URL pública
```

O frontend (build estático do Vite) é publicado num bucket S3 com hospedagem de site estático, com uma distribuição CloudFront na frente para fornecer HTTPS.

Documentação da API

A especificação segue o padrão OpenAPI 3.0 (Swagger) e está disponível em duas formas: interface interativa (GET /docs, disponível localmente e via Docker) e especificação em JSON puro (GET /openapi.json, disponível em qualquer ambiente, incluindo a AWS Lambda). Para usar a API, é necessário se cadastrar ou fazer login (retorna um token), e enviar esse token no cabeçalho Authorization dos demais endpoints.

Método	Rota	Login?	O que faz
POST	/api/auth/register	Não	Cria uma nova conta de usuário
POST	/api/auth/login	Não	Autentica e retorna um token
GET	/api/auth/me	Sim	Dados do usuário autenticado
GET	/api/users/search	Sim	Busca usuário por e-mail
GET/POST	/api/categories	Sim	Lista ou cria categorias
GET/PUT/DELETE	/api/categories/{id}	Sim	Consulta, edita ou remove uma categoria
GET/POST	/api/tasks	Sim	Lista (com filtros) ou cria tarefas
GET/PUT/DELETE	/api/tasks/{id}	Sim	Consulta, edita ou remove uma tarefa
POST/DELETE	/api/tasks/{id}/collaborators	Sim	Adiciona ou remove um colaborador
GET	/api/reports/summary	Sim	Resumo agregado das tarefas
GET	/api/reports/by-category	Sim	Distribuição de tarefas por categoria
GET	/api/health	Não	Healthcheck da API

Recursos e Ambientes Publicados

Recurso	Endereço
Aplicação (frontend, HTTPS)	https://d2mdqwrxr1861a.cloudfront.net
API (backend)	https://lhz7s6kdbg.execute-api.sa-east-1.amazonaws.com
Banco de dados	PostgreSQL gerenciado (Supabase)
Repositório de código	GitHub